

Low-Level Design (LLD) - Electric Vehicle Charging Station Analysis

Difficulty Level: Medium

Technology: Pandas

Total Marks: 20

Standards Followed: 6 Methods | 6 Visible Test Cases | 2 Hidden Test Cases

Problem Statement

An electric vehicle charging network tracks charging sessions across multiple stations. Each record contains session ID, station ID, city, charging date, energy consumed, charging duration, and payment status.

Your task is to build a Pandas-based analyzer that can create charging session data, clean invalid sessions, calculate station utilization, flag long charging sessions, identify high-energy stations, and generate city-wise revenue summaries.

Class Declaration

```
class EVChargingAnalyzer:
```

Operations

1. Create Charging Sessions DataFrame

Creates a structured DataFrame from raw EV charging session records.

Function Prototype:

```
def create_sessions_df(self, session_data: list) -> pd.DataFrame:
```

Example Input:

```
analyzer = EVChargingAnalyzer()
```

```
analyzer.create_sessions_df([
    [501, "ST01", "Bengaluru", "2025-02-01", 42.5, 75, "Paid"],
    [502, "ST02", "Chennai", "2025-02-01", 28.0, 50, "Paid"],
    [503, "ST01", "Bengaluru", "2025-02-02", 55.0, 130, "Pending"]
])
```

Expected Output:

	SessionID	StationID	City	ChargingDate	EnergyKWh	DurationMinutes	PaymentStatus
0	501	ST01	Bengaluru	2025-02-01	42.5	75	Paid

1	502	ST02	Chennai	2025-02-01	28.0	50	Paid
2	503	ST01	Bengaluru	2025-02-02	55.0	130	Pending

Implementation Flow:

- Create a DataFrame using columns:
 - SessionID
 - StationID
 - City
 - ChargingDate
 - EnergyKWh
 - DurationMinutes
 - PaymentStatus
- Keep ChargingDate as a string.
- Return the DataFrame.

2. Clean Charging Sessions

Removes invalid charging session records.

Function Prototype:

```
def clean_sessions_data(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
analyzer.clean_sessions_data(df)
```

Expected Output:

Rows where station, city, energy, duration, and payment status are valid.

Implementation Flow:

- Drop rows where SessionID, StationID, City, or PaymentStatus is null.
- Keep only rows where:
 - EnergyKWh > 0
 - DurationMinutes > 0
- Keep PaymentStatus only in:

- "Paid"
- "Pending"
- "Failed"
- Return the cleaned DataFrame.

3. Add Long Session Flag

Adds a binary flag for sessions longer than the given duration threshold.

Function Prototype:

```
def add_long_session_flag(self, df: pd.DataFrame, duration_threshold: int) -> pd.DataFrame:
```

Example Input:

```
analyzer.add_long_session_flag(df, 90)
```

Expected Output:

	SessionID	StationID	City	ChargingDate	EnergyKWh	DurationMinutes	PaymentStatus	IsLongSession
0	501	ST01	Bengaluru	2025-02-01	42.5	75	Paid	0
1	502	ST02	Chennai	2025-02-01	28.0	50	Paid	0
2	503	ST01	Bengaluru	2025-02-02	55.0	130	Pending	1

Implementation Flow:

- Create column IsLongSession.
 - Set value as 1 if DurationMinutes > duration_threshold.
 - Otherwise, set value as 0.
 - Return the updated DataFrame.
-

4. Station Utilization Summary

Calculates session count, total energy, and average duration per station.

Function Prototype:

```
def station_utilization_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
analyzer.station_utilization_summary(df)
```

Expected Output:

	StationID	SessionCount	TotalEnergyKWh	AverageDuration
0	ST01	2	97.5	102.5
1	ST02	1	28.0	50.0

Implementation Flow:

- Group by StationID.
 - Calculate:
 - SessionCount as count of sessions
 - TotalEnergyKWh as sum of EnergyKWh
 - AverageDuration as mean of DurationMinutes
 - Round AverageDuration to 1 decimal.
 - Reset index and return the DataFrame.
-

5. High Energy Stations

Returns stations whose total energy consumption is greater than the given threshold.

Function Prototype:

```
def high_energy_stations(self, df: pd.DataFrame, energy_threshold: float) -> pd.DataFrame:
```

Example Input:

```
analyzer.high_energy_stations(df, 80.0)
```

Expected Output:

	StationID	TotalEnergyKWh
0	ST01	97.5

Implementation Flow:

- Group by StationID.
- Sum EnergyKWh.

- Rename the output column as TotalEnergyKWh.
- Return only rows where TotalEnergyKWh > energy_threshold.
- Use strict greater than comparison.

6. City Revenue Summary

Calculates city-wise revenue from paid charging sessions.

Assume revenue is calculated as:

Revenue = EnergyKWh * 18

Function Prototype:

```
def city_revenue_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
analyzer.city_revenue_summary(df)
```

Expected Output:

```

      City Revenue
0  Bengaluru   765.0
1   Chennai   504.0

```

Implementation Flow:

- Filter rows where PaymentStatus == "Paid".
- Create revenue using EnergyKWh * 18.
- Group by City.
- Sum revenue.
- Rename output column as Revenue.
- Reset index and return the DataFrame.

Test Cases and Marks Allocation

Test Case ID	Type	Method	Validation Focus	Marks
--------------	------	--------	------------------	-------

TC1	Visible	create_sessions_df()	Creates DataFrame with correct schema	2.5
TC2	Visible	clean_sessions_data()	Removes invalid/null records	2.5
TC3	Visible	add_long_session_flag()	Adds correct binary flag	2.5
TC4	Visible	station_utilization_summary()	Aggregates count, energy, and average duration	3
TC5	Visible	high_energy_stations()	Filters stations using strict threshold	3
TC6	Visible	city_revenue_summary()	Calculates revenue only for paid sessions	3
HTC1	Hidden	clean_sessions_data()	Handles zero/negative energy and invalid payment status	1.5
HTC2	Hidden	high_energy_stations()	Excludes stations equal to threshold	2